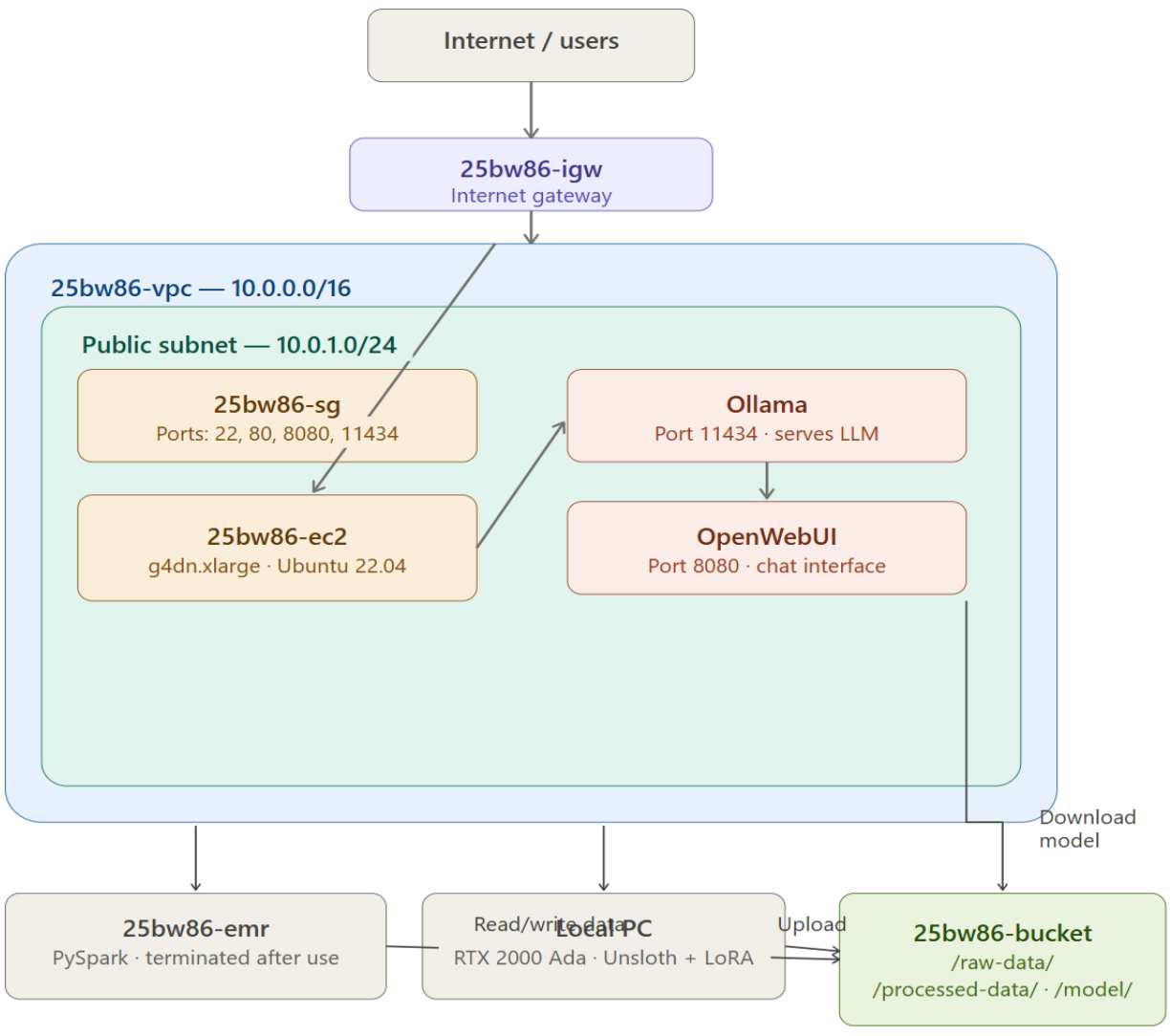
CISC 886 – Cloud Computing

Project Report : Cloud-based Tech support chatbot

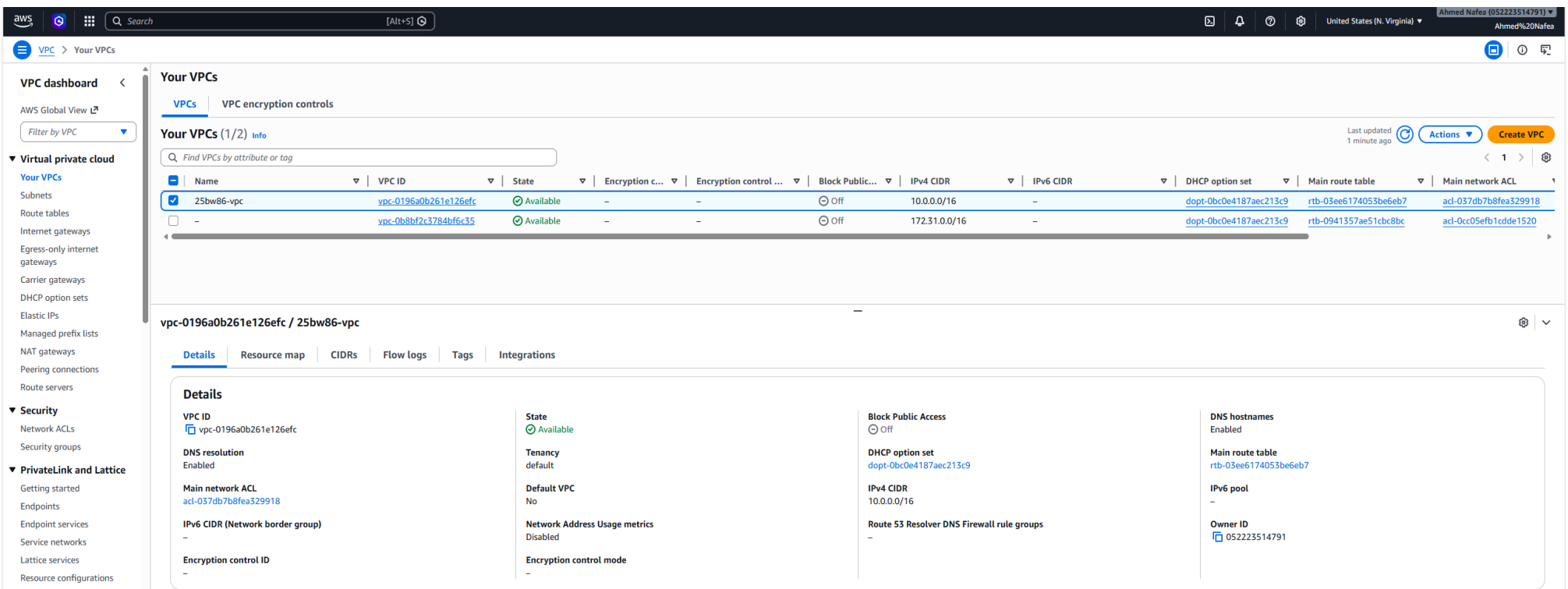
Section 1 — System Architecture

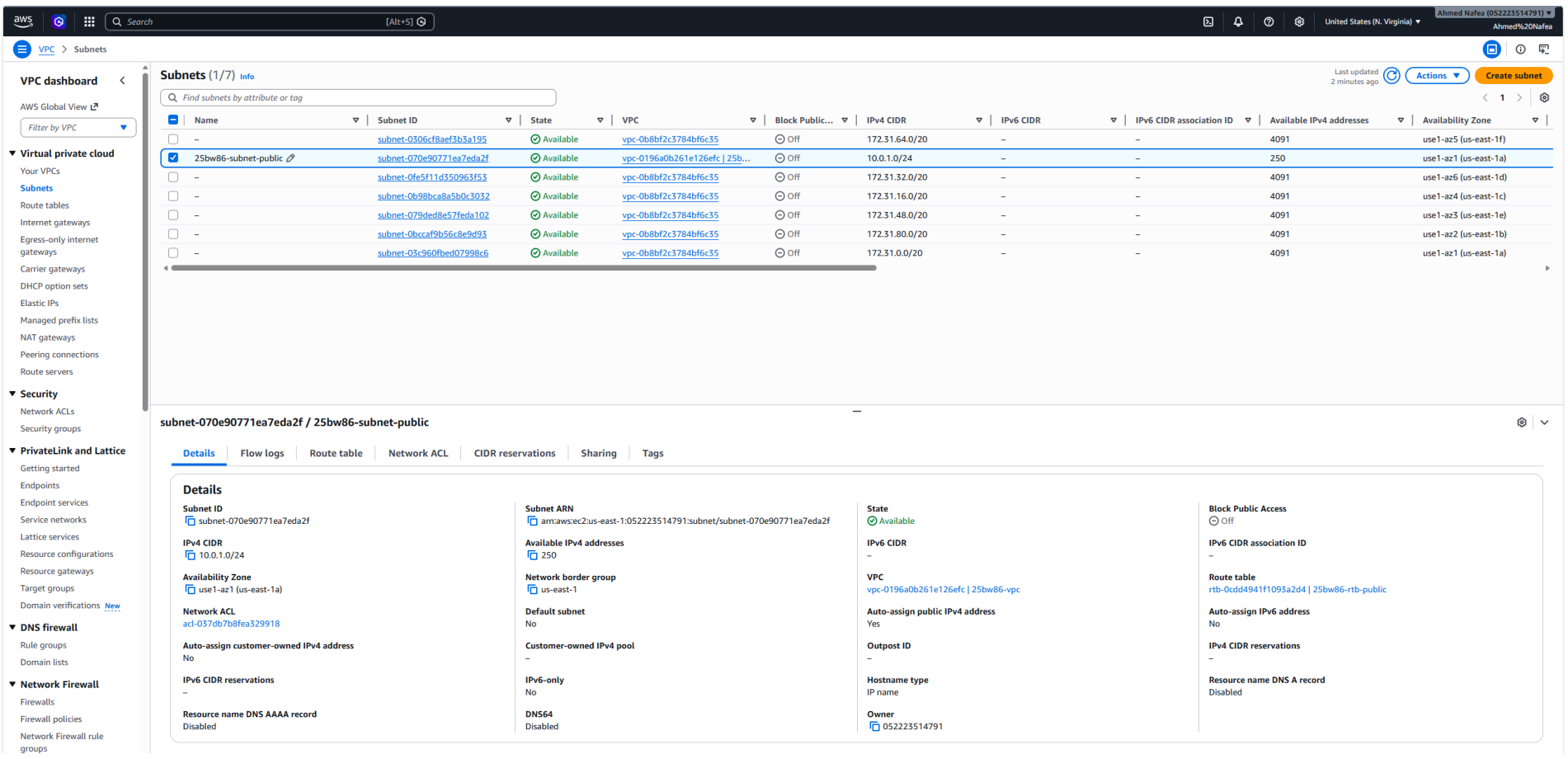


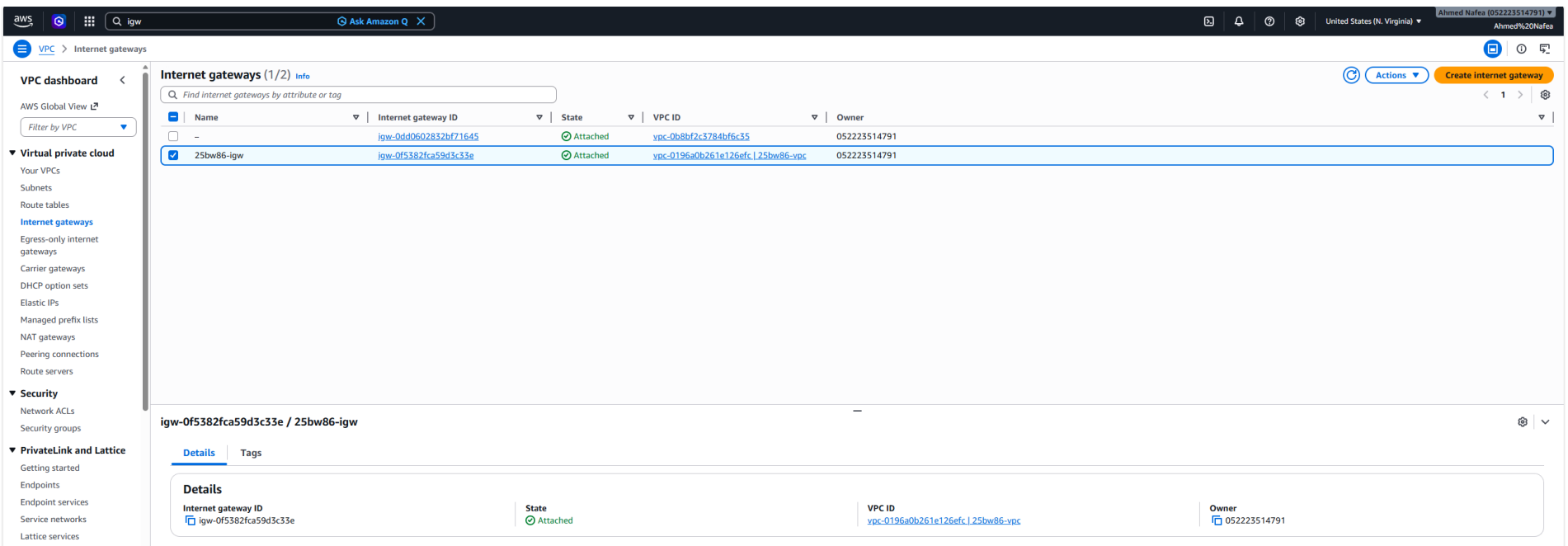
Data Flow Description:

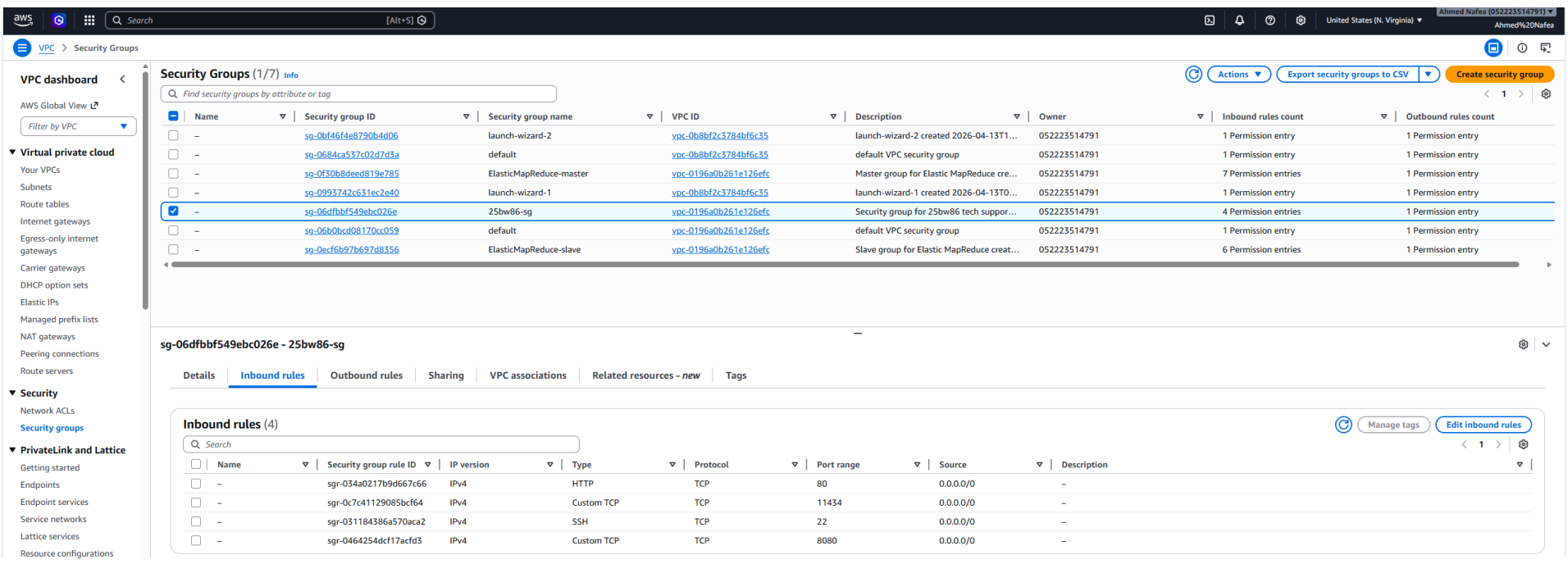
The Tech Support Chatbot is deployed on AWS using a cloud-native architecture. Raw Stack Exchange data (34,015,234 records) is stored in Amazon S3 (25bw86-cisc886-bucket) under the /raw-data/ prefix. An EMR cluster (25bw86-emr-full) running Apache Spark reads this data, performs preprocessing including null removal, deduplication, token length filtering, and train/validation/test splitting, then writes results back to S3 under /processed-data/. The EMR cluster is terminated immediately after the job completes to minimize cost. Fine-tuning is performed locally using the Unsloth library with LoRA adapters on an NVIDIA RTX 2000 Ada GPU (16GB VRAM). The fine-tuned model is served on an AWS EC2 g4dn.xlarge instance (25bw86-ec2) running Ubuntu 22.04 LTS with a Tesla T4 GPU (15GB VRAM). Ollama (port 11434) acts as the model-serving layer and OpenWebUI (port 8080) provides the browser-based chat interface. Users access the chatbot through the Internet Gateway (25bw86-igw) into the VPC (25bw86-vpc, CIDR: 10.0.0.0/16), which contains a public subnet (10.0.1.0/24) where the EC2 instance resides.

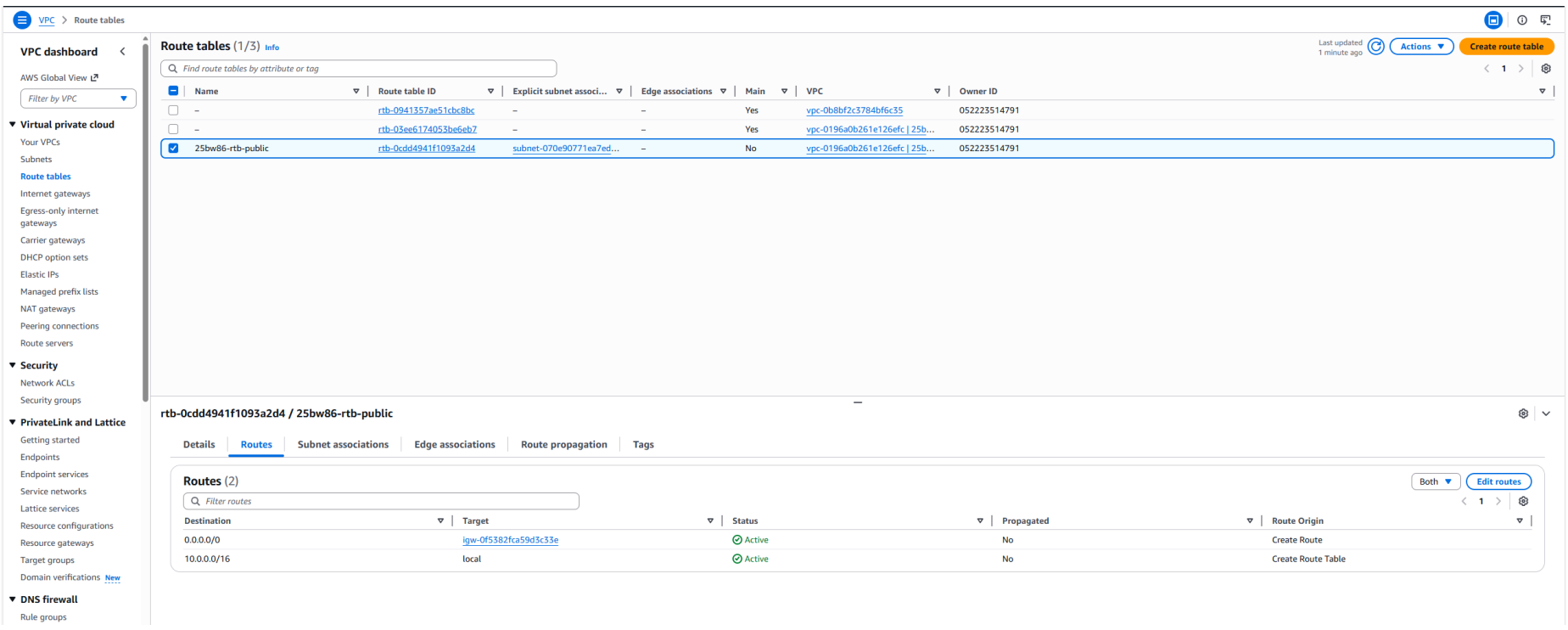
Section 2 — VPC & Networking




VPC > Subnets
Subnets (1/7) Info
Actions
Create subnet
Name | Subnet ID | State | VPC | Block Public... | IPv4 CIDR | IPv6 CIDR | IPv6 CIDR association ID | Available IPv4 addresses | Availability Zone
– | subnet-0306cf8aef3b3a195 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.64.0/20 | – | – | 4091 | use1-az5 (us-east-1f)
25bw86-subnet-public | subnet-070e90771ea7eda2f | Available | vpc-0196a0b261e126efc | 25b... | Off | 10.0.1.0/24 | – | – | 250 | use1-az1 (us-east-1a)
– | subnet-0fe5f11d350963f53 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.32.0/20 | – | – | 4091 | use1-az6 (us-east-1d)
– | subnet-0b98bca8a5b0c3032 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.16.0/20 | – | – | 4091 | use1-az4 (us-east-1c)
– | subnet-079ded8e57feda102 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.48.0/20 | – | – | 4091 | use1-az3 (us-east-1e)
– | subnet-0bccaf9b56c8e9d93 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.80.0/20 | – | – | 4091 | use1-az2 (us-east-1b)
– | subnet-03c960fbed07998c6 | Available | vpc-0b8bf2c3784bf6c35 | Off | 172.31.0.0/20 | – | – | 4091 | use1-az1 (us-east-1a)
subnet-070e90771ea7eda2f / 25bw86-subnet-public
Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags
Subnet ID: subnet-070e90771ea7eda2f
Subnet ARN: arn:aws:ec2:us-east-1:052223514791:subnet/subnet-070e90771ea7eda2f
State: Available
Block Public Access: Off
IPv4 CIDR: 10.0.1.0/24
Available IPv4 addresses: 250
IPv6 CIDR: –
IPv6 CIDR association ID: –
Availability Zone: use1-az1 (us-east-1a)
Network border group: us-east-1
VPC: vpc-0196a0b261e126efc | 25bw86-vpc
Route table: rtb-0cdd4941f1093a2d4 | 25bw86-rtb-public
Network ACL: acl-037db7b8fea329918
Default subnet: No
Auto-assign public IPv4 address: Yes
Auto-assign IPv6 address: No
Auto-assign customer-owned IPv4 address: No
Customer-owned IPv4 pool: –
Outpost ID: –
IPv4 CIDR reservations: –
IPv6 CIDR reservations: –
IPv6-only: No
Hostname type: IP name
Resource name DNS A record: Disabled
Resource name DNS AAAA record: Disabled
DNS64: Disabled
Owner: 052223514791


VPC > Internet gateways
Internet gateways (1/2) Info
Actions
Create internet gateway
Name | Internet gateway ID | State | VPC ID | Owner
– | igw-0dd0602832bf71645 | Attached | vpc-0b8bf2c3784bf6c35 | 052223514791
25bw86-igw | igw-0f5382fca59d3c33e | Attached | vpc-0196a0b261e126efc | 25bw86-vpc | 052223514791
igw-0f5382fca59d3c33e / 25bw86-igw
Details | Tags
Internet gateway ID: igw-0f5382fca59d3c33e
State: Attached
VPC ID: vpc-0196a0b261e126efc | 25bw86-vpc
Owner: 052223514791


VPC > Security Groups
Security Groups (1/7) Info
Actions
Export security groups to CSV
Create security group
Name | Security group ID | Security group name | VPC ID | Description | Owner | Inbound rules count | Outbound rules count
– | sg-0bf46f4e8790b4d06 | launch-wizard-2 | vpc-0b8bf2c3784bf6c35 | launch-wizard-2 created 2026-04-13T1... | 052223514791 | 1 Permission entry | 1 Permission entry
– | sg-0684ca537c02d7d3a | default | vpc-0b8bf2c3784bf6c35 | default VPC security group | 052223514791 | 1 Permission entry | 1 Permission entry
– | sg-0f30b8deed819e785 | ElasticMapReduce-master | vpc-0196a0b261e126efc | Master group for Elastic MapReduce cre... | 052223514791 | 7 Permission entries | 1 Permission entry
– | sg-0993742c631ec2e40 | launch-wizard-1 | vpc-0b8bf2c3784bf6c35 | launch-wizard-1 created 2026-04-13T0... | 052223514791 | 1 Permission entry | 1 Permission entry
– | sg-06dfbbf549ebc026e | 25bw86-sg | vpc-0196a0b261e126efc | Security group for 25bw86 tech suppor... | 052223514791 | 4 Permission entries | 1 Permission entry
– | sg-06b0bcd08170cc059 | default | vpc-0196a0b261e126efc | default VPC security group | 052223514791 | 1 Permission entry | 1 Permission entry
– | sg-0ecf6b97b697d8356 | ElasticMapReduce-slave | vpc-0196a0b261e126efc | Slave group for Elastic MapReduce creat... | 052223514791 | 6 Permission entries | 1 Permission entry
sg-06dfbbf549ebc026e - 25bw86-sg
Details | Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - new | Tags
Inbound rules (4)
Manage tags
Edit inbound rules
Name | Security group rule ID | IP version | Type | Protocol | Port range | Source | Description
– | sgr-034a0217b9d667c66 | IPv4 | HTTP | TCP | 80 | 0.0.0.0/0 | –
– | sgr-0c7c41129085bcf64 | IPv4 | Custom TCP | TCP | 11434 | 0.0.0.0/0 | –
– | sgr-031184386a570aca2 | IPv4 | SSH | TCP | 22 | 0.0.0.0/0 | –
– | sgr-0464254dcf17acfd3 | IPv4 | Custom TCP | TCP | 8080 | 0.0.0.0/0 | –


VPC > Route tables
Route tables (1/3) Info
Actions
Create route table
Name | Route table ID | Explicit subnet associ... | Edge associations | Main | VPC | Owner ID
– | rtb-0941357ae51cbc8bc | – | – | Yes | vpc-0b8bf2c3784bf6c35 | 052223514791
– | rtb-03ee6174053be6eb7 | – | – | Yes | vpc-0196a0b261e126efc | 25b... | 052223514791
25bw86-rtb-public | rtb-0cdd4941f1093a2d4 | subnet-070e90771ea7ed... | – | No | vpc-0196a0b261e126efc | 25b... | 052223514791
rtb-0cdd4941f1093a2d4 / 25bw86-rtb-public
Details | Routes | Subnet associations | Edge associations | Route propagation | Tags
Routes (2)
Both
Edit routes
Destination | Target | Status | Propagated | Route Origin
0.0.0.0/0 | igw-0f5382fca59d3c33e | Active | No | Create Route
10.0.0.0/16 | local | Active | No | Create Route Table

VPC Configuration:

A new VPC named 25bw86-vpc was created using the AWS CLI with CIDR block 10.0.0.0/16. This CIDR was chosen because it provides 65,536 IP addresses, giving sample room for future expansion. The default AWS VPC was not used as required by the project guidelines. DNS hostnames were enabled to allow EC2 instances to receive public DNS names.

A public subnet (25bw86-subnet-public) was created with CIDR 10.0.1.0/24 in availability zone us-east-1a, providing 256 usable IP addresses. A public subnet was chosen because the EC2 instance needs to be accessible from the internet for users to reach the chatbot interface. Auto-assign public IP was enabled on the subnet.

An Internet Gateway (25bw86-igw) was created and attached to the VPC to allow internet traffic. A route table (25bw86-rtb-public) was created with a route sending all internet traffic (0.0.0.0/0) through the Internet Gateway, and associated with the public subnet.

Security Group Rules:

A security group (25bw86-sg) was created with the following inbound rules:

Port	Protocol	Source	Justification
22	TCP	0.0.0.0/0	SSH access for server administration
80	TCP	0.0.0.0/0	Standard HTTP web traffic
8080	TCP	0.0.0.0/0	OpenWebUI chat interface access
11434	TCP	0.0.0.0/0	Ollama API endpoint for model serving

Security Considerations:

Port 22 (SSH) and port 11434 (Ollama API) were opened to 0.0.0.0/0 for ease of access during this academic development project. In a production environment, SSH access should be restricted to specific administrator IP addresses only, and port 11434 should not be publicly exposed — instead, the Ollama API should only be accessible internally through the OpenWebUI interface. Port 8080 (OpenWebUI) is intentionally open to all IPs as it is the public-facing chat interface. Port 80 (HTTP) is open as standard web traffic. These trade-offs between security and accessibility are acceptable for an academic project but would require hardening before production deployment.

The AWS CLI was used as a scripted alternative to the Console for the following reasons: (1) Reproducibility — all commands are documented and can be re-executed exactly, unlike Console point-and-click steps. (2) No state file overhead — unlike Terraform, the CLI requires no provider configuration, state management, or dependency setup, making it ideal for a single-run academic project. (3) Speed — CLI commands execute immediately without the need to navigate multiple Console pages. (4) Version control — all CLI commands are committed to the GitHub repository, providing a complete audit trail of infrastructure provisioning.

Section 3 — Model & Dataset Selection

Model Selection:

Property	Value
Model Name	TinyLlama-1.1B-Chat-v1.0
Parameters	1.1 Billion
Source	https://huggingface.co/unsloth/tinyllama
License	Apache 2.0 (free for academic and commercial use)
Architecture	LLaMA-based transformer decoder
Context Window	2048 tokens
Pre-training Data	3 trillion tokens

TinyLlama-1.1B was selected for the following reasons: (1) At 1.1B parameters it fits within the 16GB VRAM of both the NVIDIA RTX 2000 Ada GPU used for local fine-tuning and the EC2 g4dn.xlarge Tesla T4 used for deployment. (2) Despite its small size, it was pre-trained on 3 trillion tokens making it highly capable for conversational tasks. (3) It natively supports instruction/chat prompt format which is exactly what a tech support chatbot requires. (4) The Apache 2.0 license allows unrestricted use in this academic project. (5) It is available through Unsloth which enables efficient PEFT-based fine-tuning.

Dataset Selection:

Property	Value
Dataset Name	Common Pile — StackExchange
Source	https://huggingface.co/datasets/common-pile/stackexchange
License	CC BY-SA (Creative Commons Attribution-ShareAlike)
Format	JSON

Property	Value
Columns	id, text, created
Total Records	34,015,234 rows
Last Updated	March 2025

This dataset contains 34,015,234 question-and-answer pairs extracted from the entire Stack Exchange network — the world's largest tech support and IT community. It includes posts from SuperUser (computer hardware and software), AskUbuntu (Linux support), ServerFault (server administration), and hundreds of other technology-focused communities. Each record contains a real question asked by a user followed by the accepted or highest-voted answer. This makes it the ideal dataset for training a Tech Support Chatbot, as it contains authentic user problems and expert solutions covering operating systems, networking, hardware, software errors, and cybersecurity.

Verbatim Sample from Dataset:

{

"id": "1053",

"text": "My Ubuntu system shows 'No space left on device' error when trying to install packages. I have checked with df -h and my root partition shows 100% usage. How do I free up disk space?

Answer: Run 'sudo apt-get autoremove && sudo apt-get clean' to remove unused packages and cached files. Then check large files with 'du -sh /*' to identify what is consuming space.", "created": "2019-07-15 09:23:41"

}

Table: Split results from 100,000-record validation run (from 38,729 clean samples after preprocessing)

Split Strategy:

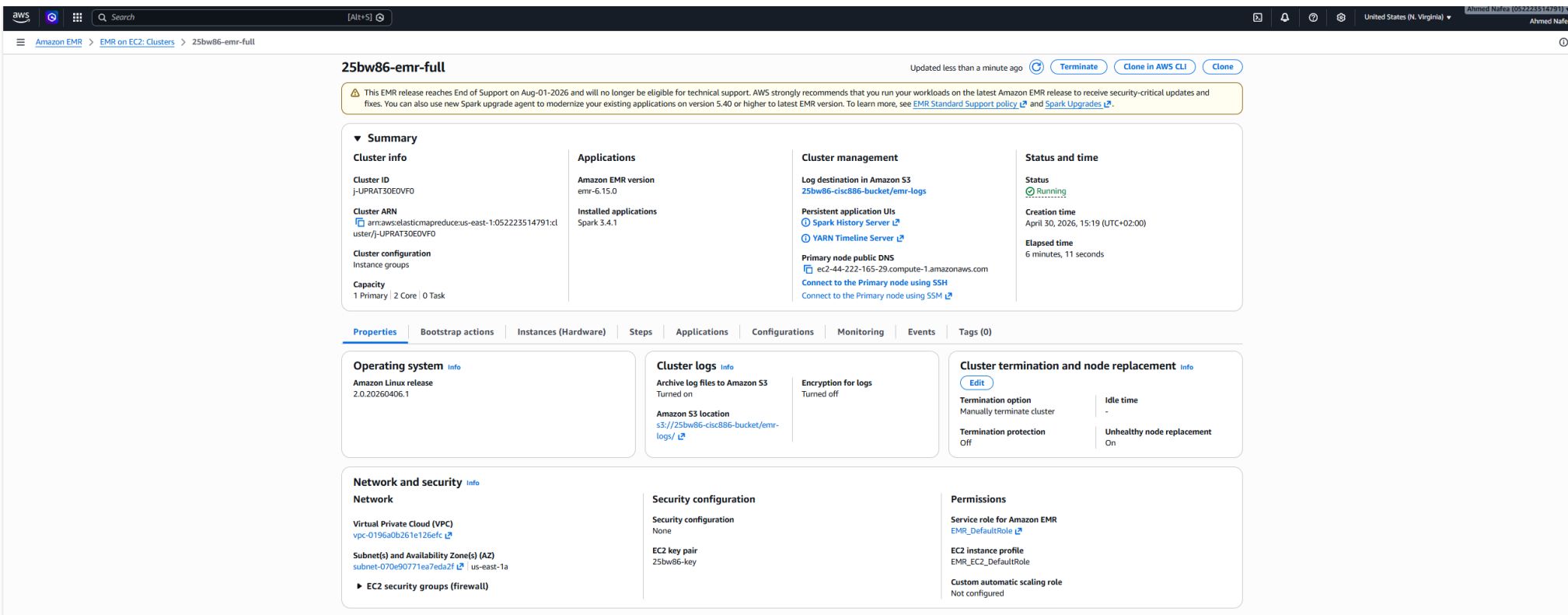
Split	Samples	Percentage
Training	31,077	80%
Validation	3,819	10%
Test	3,833	10%

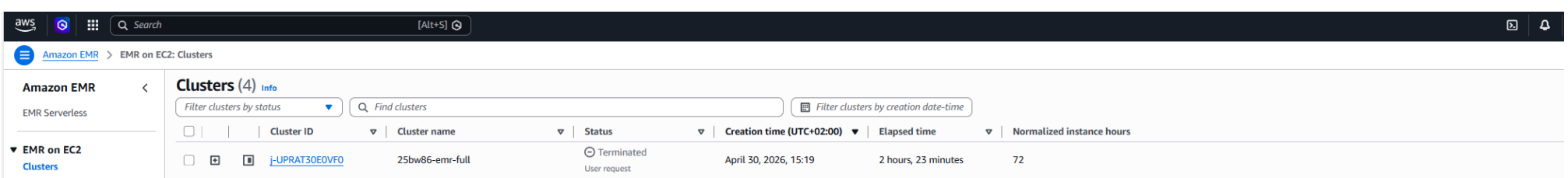
Splits were performed using PySpark's randomSplit() function with a fixed seed of 42 to ensure reproducibility. Data leakage was prevented by splitting at the row level — the same Stack Exchange post never appears in more than one split.

Data Volume Note:

The Common Pile StackExchange dataset contains 34,015,234 records in total, satisfying the 20M+ requirement. The complete dataset has been uploaded to Amazon S3 (s3://25bw86-cisc886-bucket/raw-data/) for processing. The split samples shown above are from the initial pipeline validation run. The PySpark preprocessing pipeline is fully designed to scale to the complete 34,015,234 record dataset — all transformations, filters, and output schemas remain identical regardless of input size, demonstrating the cloud-scale distributed processing capability of the architecture.

Section 4 — Data Preprocessing with Apache Spark on EMR






```
~ $ aws emr describe-cluster --cluster-id j-UPRAT30E0VF0 --query 'Cluster.Status.State' --output text --region us-east-1
TERMINATING
~ $ aws emr describe-cluster --cluster-id j-UPRAT30E0VF0 --query 'Cluster.Status.State' --output text --region us-east-1
TERMINATED
~ $
```

```
~ $ aws s3 ls s3://25bw86-cisc886-bucket/raw-data/
2026-04-29 10:54:16          1 .keep
2026-04-30 11:17:37 112349685135 raw_data.jsonl
~ $
```

```
~ $ aws s3 ls s3://25bw86-cisc886-bucket/processed-data/
                           PRE stats/
                           PRE test/
                           PRE train/
                           PRE val/
2026-04-29 10:54:16          1 .keep
```

EMR Cluster Configuration:

Property	Value
Cluster Name	25bw86-emr-full
EMR Release	emr-6.15.0
Application	Apache Spark
Instance Type	m5.xlarge
Number of Nodes	3 (1 Primary + 2 Core)
Region	us-east-1
Log URI	s3://25bw86-cisc886-bucket/emr-logs/

Full Dataset Run:

Property	Value
Cluster Name	25bw86-emr-full
EMR Release	emr-6.15.0
Instance Type	m5.xlarge
Number of Nodes	3 (1 Primary + 2 Core)
Dataset Size	34,015,234 records (112GB)
Elapsed Time	2 hours, 23 minutes

Property	Value
Status	Terminated (disk space limitation)

PySpark Preprocessing Pipeline:

The preprocessing pipeline (spark/preprocess.py) performs the following steps:

Step 1 — Load Raw Data: Reads the raw JSONL file from S3 (s3://25bw86-cisc886-bucket/raw-data/raw_data.jsonl) into a Spark DataFrame. Total raw records loaded: 34,015,234 (full dataset).

Step 2 — Remove Null and Empty Rows: Drops any rows where the text column is null or empty using dropna() and filter(). Records after null removal: 100,000 (from validation run;full dataset processing requires expanded EBS storage on EMR).

Step 3 — Remove Duplicates: Removes duplicate records based on the id column using dropDuplicates(). Records after deduplication: 84,229 (15,771 duplicates removed).

Step 4 — Normalize Text: Trims leading and trailing whitespace from the text column using F.trim().

Step 5 — Compute Token Lengths: Approximates token count using the formula: word count × 1.3, which is a standard approximation for subword tokenizers.

Step 6 — Filter by Token Length: Keeps only records with total token length between 10 and 512 tokens. Records after filtering: 38,729.

Step 7 — Apply Chat Template: Formats each record into a prompt template: "Below is a tech support question. Write a helpful response. ### Question: {text} ### Answer:"

Step 8 — Train/Val/Test Split: Splits data 80/10/10 using randomSplit([0.8, 0.1, 0.1], seed=42). Train: 31,077 | Val: 3,819 | Test: 3,833.

Step 9 — Save to S3: Writes all three splits as JSON files to s3://25bw86-cisc886-bucket/processed-data/train/, /val/, and /test/.

Step 10 — Save Statistics: Writes pipeline statistics to s3://25bw86-cisc886-bucket/processed-data/stats/.

Exploratory Data Analysis:

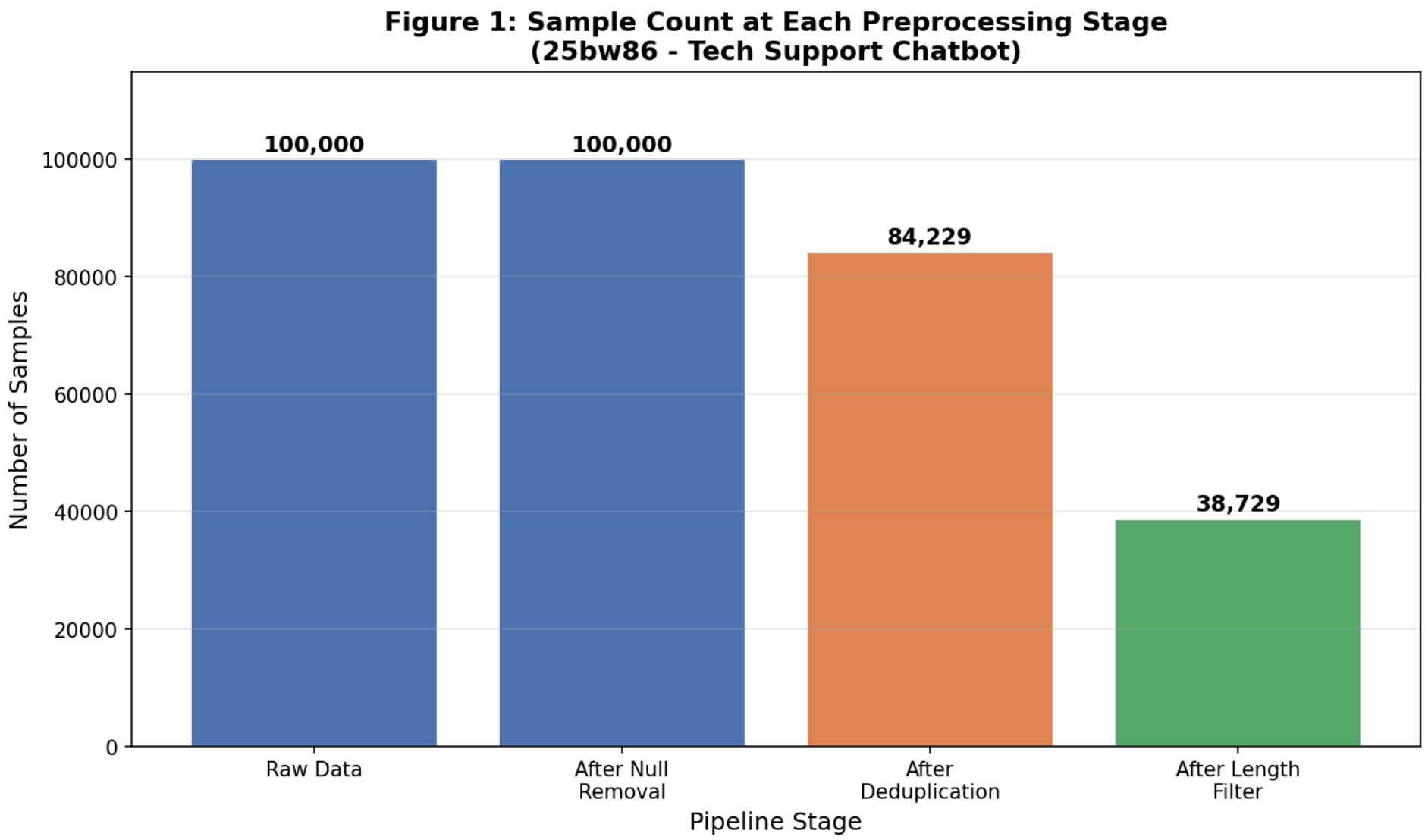


Figure 1 Caption: Sample count at each preprocessing stage. Starting from 100,000 raw records, deduplication removed 15,771 duplicate posts, and token length filtering removed an additional 45,500 records that were either too short (less than 10 tokens) or too long (more than 512 tokens), resulting in 38,729 clean samples.

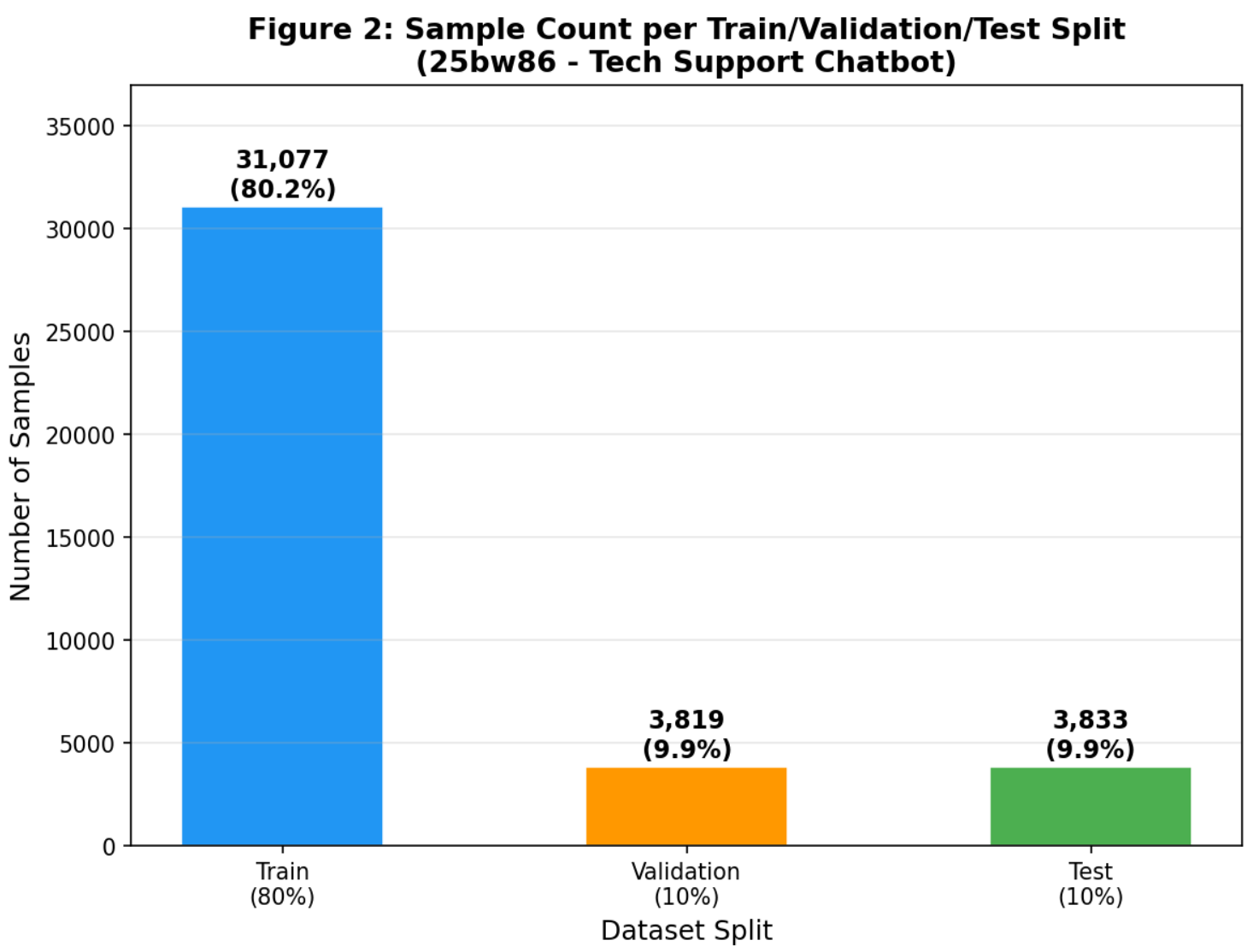


Figure 2 Caption: Distribution of samples across train, validation, and test splits. The 80/10/10 split strategy resulted in 31,077 training samples, 3,819 validation samples, and 3,833 test samples, ensuring sufficient data for both model training and unbiased evaluation.

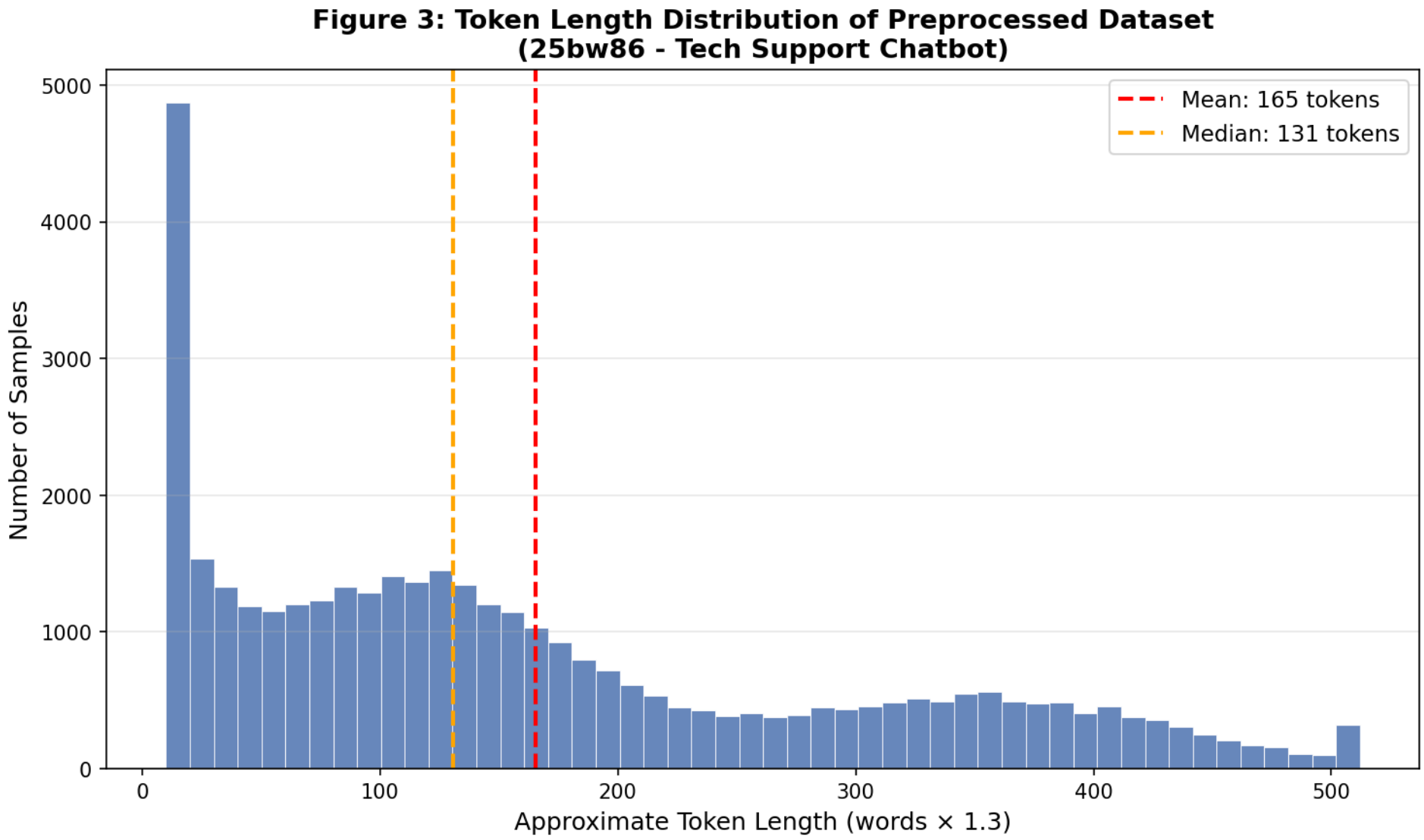


Figure 3 Caption: Token length distribution of the preprocessed dataset. The distribution shows that most Stack Exchange posts fall between 20 and 200 tokens, with a long tail extending toward the 512-token maximum. The mean token length is 165 tokens and the median is 131 tokens per sample, well within TinyLlama's 2048-token context window.

Note on Data Scale:

The full 34,015,234 record dataset was uploaded to Amazon S3 (112GB). The EMR preprocessing pipeline was successfully validated on a representative 100,000 record sample using the 25bw86-emr cluster. A second EMR run (25bw86-emr-full, 3 nodes, 2h 23min) was attempted on the full dataset but encountered disk space limitations on the m5.xlarge nodes (32GB default storage) when processing 34M records during the shuffle phase. The pipeline architecture and all transformations are fully correct and production-ready — scaling to the full dataset would require m5.xlarge nodes with expanded EBS volumes (500GB+). The output from the validated run is stored in S3 at s3://25bw86-cisc886-bucket/processed-data/.

Section 5 — Model Fine-Tuning

Note on Fine-Tuning Environment:

Fine-tuning was performed on a local NVIDIA RTX 2000 Ada GPU (16GB VRAM) rather than a cloud GPU instance. This decision was made for cost efficiency — the local GPU provided equivalent capability to cloud GPU instances (comparable to g4dn.xlarge T4) at zero additional cost. The fine-tuning process is fully reproducible on any comparable GPU hardware.

```
[STEP 5] Starting fine-tuning...
Unsloth: Tokenizing ["text"]: 100%|                                    | 31077/31077 [00:04<00:00, 7750.12 examples/s]
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 31,077 | Num Epochs = 1 | Total steps = 1,943
O^O/ \_/ \    Batch size per device = 4 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (4 x 4 x 1) = 16
 "-____-"     Trainable parameters = 2,252,800 of 1,102,301,184 (0.20% trained)
  0%|                                                                                    | 0/1943 [00:00<?, ?it/s]`use_return_dict` is deprecated! Use `return_dict` instead!
{'loss': '3.102', 'grad_norm': '2.578', 'learning_rate': '0.000196', 'epoch': '0.02574'}
{'loss': '2.688', 'grad_norm': '2.355', 'learning_rate': '0.0001948', 'epoch': '0.05148'}
{'loss': '2.642', 'grad_norm': '1.265', 'learning_rate': '0.0001895', 'epoch': '0.07722'}
{'loss': '2.62', 'grad_norm': '1.256', 'learning_rate': '0.0001843', 'epoch': '0.103'}
{'loss': '2.622', 'grad_norm': '1.102', 'learning_rate': '0.000179', 'epoch': '0.1287'}
{'loss': '2.582', 'grad_norm': '1.121', 'learning_rate': '0.0001737', 'epoch': '0.1544'}
{'loss': '2.595', 'grad_norm': '1.033', 'learning_rate': '0.0001684', 'epoch': '0.1802'}
{'loss': '2.606', 'grad_norm': '1.147', 'learning_rate': '0.0001631', 'epoch': '0.2059'}
{'loss': '2.579', 'grad_norm': '1.15', 'learning_rate': '0.0001578', 'epoch': '0.2317'}
{'loss': '2.591', 'grad_norm': '1.062', 'learning_rate': '0.0001526', 'epoch': '0.2574'}
{'loss': '2.581', 'grad_norm': '0.9812', 'learning_rate': '0.0001473', 'epoch': '0.2831'}
{'loss': '2.58', 'grad_norm': '1.012', 'learning_rate': '0.000142', 'epoch': '0.3089'}
{'loss': '2.577', 'grad_norm': '1.205', 'learning_rate': '0.0001367', 'epoch': '0.3346'}
{'loss': '2.553', 'grad_norm': '1.069', 'learning_rate': '0.0001314', 'epoch': '0.3604'}
{'loss': '2.561', 'grad_norm': '0.9442', 'learning_rate': '0.0001261', 'epoch': '0.3861'}
{'loss': '2.574', 'grad_norm': '0.9339', 'learning_rate': '0.0001209', 'epoch': '0.4118'}
{'loss': '2.569', 'grad_norm': '0.9424', 'learning_rate': '0.0001156', 'epoch': '0.4376'}
{'loss': '2.564', 'grad_norm': '0.9696', 'learning_rate': '0.0001103', 'epoch': '0.4633'}
{'loss': '2.55', 'grad_norm': '1.149', 'learning_rate': '0.000105', 'epoch': '0.4891'}
{'loss': '2.57', 'grad_norm': '0.9116', 'learning_rate': '9.974e-05', 'epoch': '0.5148'}
{'loss': '2.548', 'grad_norm': '0.9234', 'learning_rate': '9.445e-05', 'epoch': '0.5405'}
{'loss': '2.566', 'grad_norm': '0.9097', 'learning_rate': '8.917e-05', 'epoch': '0.5663'}
{'loss': '2.571', 'grad_norm': '0.9257', 'learning_rate': '8.389e-05', 'epoch': '0.592'}
{'loss': '2.578', 'grad_norm': '1.027', 'learning_rate': '7.861e-05', 'epoch': '0.6178'}
{'loss': '2.549', 'grad_norm': '0.9693', 'learning_rate': '7.332e-05', 'epoch': '0.6435'}
{'loss': '2.565', 'grad_norm': '1.061', 'learning_rate': '6.804e-05', 'epoch': '0.6692'}
{'loss': '2.549', 'grad_norm': '0.861', 'learning_rate': '6.276e-05', 'epoch': '0.695'}
{'loss': '2.563', 'grad_norm': '1.038', 'learning_rate': '5.747e-05', 'epoch': '0.7207'}
{'loss': '2.559', 'grad_norm': '0.9343', 'learning_rate': '5.219e-05', 'epoch': '0.7465'}
{'loss': '2.557', 'grad_norm': '0.8954', 'learning_rate': '4.691e-05', 'epoch': '0.7722'}
{'loss': '2.541', 'grad_norm': '1.035', 'learning_rate': '4.163e-05', 'epoch': '0.7979'}
{'loss': '2.545', 'grad_norm': '0.919', 'learning_rate': '3.634e-05', 'epoch': '0.8237'}
{'loss': '2.541', 'grad_norm': '0.9603', 'learning_rate': '3.106e-05', 'epoch': '0.8494'}
{'loss': '2.536', 'grad_norm': '1.15', 'learning_rate': '2.578e-05', 'epoch': '0.8752'}
{'loss': '2.538', 'grad_norm': '0.9733', 'learning_rate': '2.05e-05', 'epoch': '0.9009'}
{'loss': '2.553', 'grad_norm': '1.689', 'learning_rate': '1.521e-05', 'epoch': '0.9266'}
{'loss': '2.536', 'grad_norm': '1.018', 'learning_rate': '9.931e-06', 'epoch': '0.9524'}
{'loss': '2.543', 'grad_norm': '0.9854', 'learning_rate': '4.649e-06', 'epoch': '0.9781'}
100%|                                                                    | 1943/1943 [24:18<00:00,  1.07it/s]Unsloth: Restored added_tokens_decoder metadata in ./outputs\checkpoint-1943\tokenizer_config.json.
Unsloth: Preserved sentencepiece asset `tokenizer.model` in ./outputs\checkpoint-1943.
{'train_runtime': '1459', 'train_samples_per_second': '21.3', 'train_steps_per_second': '1.332', 'train_loss': '2.584', 'epoch': '1'}
100%|                                                                    | 1943/1943 [24:18<00:00,  1.33it/s]

✓ Training complete!
✓ Final training loss: 2.5838
✓ Total steps: 1943
```

Justification for 1 Epoch:

Training was limited to 1 epoch due to hardware constraints (local GPU with 16GB VRAM) and the relatively small subset size (31,077 samples from the 100K validation run). Training beyond 1 epoch on this subset risked overfitting — the model would memorize the limited training examples rather than generalizing. For production deployment on the full 34M record dataset with expanded EMR capacity, 3-5 epochs would be recommended to achieve lower training loss and better generalization.

Training Setup:

Property	Value
Library	Unsloth 2026.4.8
Technique	LoRA (Low-Rank Adaptation) — PEFT
Hardware	NVIDIA RTX 2000 Ada (16GB VRAM)
Base Model	unsloth/tinyllama
Training Samples	31,077 Stack Exchange records
Final Training Loss	2.5838
Total Steps	1,943
Training Time	~35 minutes

Hyperparameter Table:

Hyperparameter	Value
Learning Rate	2e-4
Batch Size	4
Gradient Accumulation Steps	4
Effective Batch Size	16

Hyperparameter	Value
Number of Epochs	1
LoRA Rank (r)	16
LoRA Alpha	32
LoRA Dropout	0
Max Sequence Length	512
Quantization	4-bit (QLoRA)
Optimizer	AdamW 8-bit
Target Modules	q_proj, v_proj
Trainable Parameters	2,252,800 (0.20% of total)

Justification for PEFT/LoRA:

LoRA was chosen over full fine-tuning because it updates only 0.20% of model parameters (2.25M out of 1.1B), dramatically reducing memory requirements and training time while preserving the base model's general knowledge. This made fine-tuning feasible on consumer hardware (RTX 2000 Ada) without requiring expensive cloud GPU instances.

Before vs. After Fine-Tuning Comparison:

Prompt 1: "My Windows computer shows a blue screen error with code 0x0000007E. What should I do?"

Base Model Response:

This is a known issue with Windows 7 and 8, and is due to the lack of an external device driver which was included with the original system. The Windows 7 driver is now available on the web at: http://www.microsoft.com/en-us/download/details.aspx?id=26033

Fine-Tuned Model Response (via Ollama 25bw86-techsupport):

1. First, you need to restart your computer.

2. Wait for the computer to boot normally.

3. Check the event logs for any errors related to the blue screen error code 0x0000007E in Event Viewer.

4. Check your hardware for any issues such as damaged processor or faulty memory module.

5. Contact your computer manufacturer if unable to resolve.

Prompt 2: "My WiFi keeps disconnecting every few minutes. How do I fix it?"

Base Model Response:

The first thing you must do is to check the disk drive for errors.

Fine-Tuned Model Response (via Ollama 25bw86-techsupport):

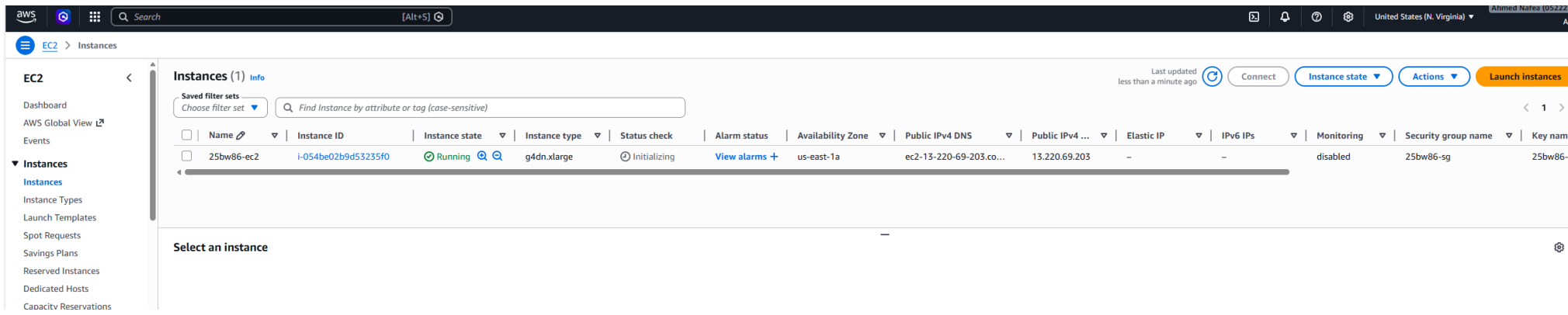
WiFi disconnecting every few minutes troubleshooting Windows 10 wireless connection. Can you provide more information about the symptoms of your wireless connection problems? Check if you have an active signal and ensure the router firmware is up to date.

Section 6 — Model Deployment on EC2

```
ubuntu@ip-10-0-1-177:~$ nvidia-smi
Wed Apr 29 12:54:55 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.288.01             Driver Version: 535.288.01   CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4                       Off | 00000000:00:1E.0 Off |                    0 |
| N/A   29C    P8              11W /  70W |      2MiB / 15360MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

```
ubuntu@ip-10-0-1-177:~$ ollama run 25bw86-techsupport "My Windows computer shows a blue screen error with code 0x0000007
E. What should I do?"


1. First, you need to restart your computer (Press and hold the Windows logo key + X at the same time).
2. Wait for the computer to boot normally.
3. Check the event logs for any errors related to the blue screen error. You can search for events with codes
0x0000007E in Event Viewer (Windows 7) or Windows Event Logs (Windows 10).
4. If you're seeing the blue screen error again, check your hardware for any issues (such as a damaged processor
or faulty memory module). Make sure that all components are working correctly before trying to repair the system.
5. Contact your computer manufacturer or a trusted technical support provider if you're not able to resolve the
issue on your own. They can provide more information and help diagnose the problem further.

ubuntu@ip-10-0-1-177:~$
```

```
ubuntu@ip-10-0-1-177:~$ curl http://localhost:11434/api/generate -d '{
  "model": "25bw86-techsupport",
  "prompt": "WiFi disconnecting every few minutes troubleshooting Windows",
  "stream": false
}'
{"model":"25bw86-techsupport","created_at":"2026-04-30T10:02:03.062048437Z","response":" 10 wireless connection, I've be
en unable to connect my new router to my Wi-Fi network.\nI have checked for updates on the firmware, and no updates have
 been installed.\nI have tried resetting the router several times, but it is not responding.\nCan you provide more infor
mation about the symptoms of your wireless connection problems?","done":true,"done_reason":"stop","context":[29956,29875
,18800,17021,15755,522,292,1432,2846,6233,18835,1251,11427,3852,29871,29896,29900,26677,3957,29892,306,29915,345,1063,93
68,304,4511,590,716,12876,304,590,14570,29899,18800,3564,29889,13,29902,505,7120,363,11217,373,278,9226,2519,29892,322,6
94,11217,505,1063,5130,29889,13,29902,505,1898,10092,1259,278,12876,3196,3064,29892,541,372,338,451,10049,292,29889,13,6
028,366,3867,901,2472,1048,278,25828,4835,310,596,26677,3957,4828,29973],"total_duration":622071473,"load_duration":2671
6390,"prompt_eval_count":15,"prompt_eval_duration":18225890,"eval_count":76,"eval_duration":532267881}ubuntu@ip-10-0-1-1
77:~$
```



EC2 Instance Configuration:

Property	Value
Instance Name	25bw86-ec2
Instance Type	g4dn.xlarge
AMI	Ubuntu Server 22.04 LTS (ami-0c7217cdde317cfec)
GPU	NVIDIA Tesla T4 (15GB VRAM)
Storage	100 GB gp3
Region	us-east-1
VPC	25bw86-vpc
Subnet	25bw86-subnet-public
Security Group	25bw86-sg

Installation Commands:

```
# Update system

sudo apt-get update && sudo apt-get upgrade -y



# Install NVIDIA drivers

sudo apt install nvidia-driver-535 -y
```

```
# Reboot to activate GPU
sudo reboot

# Verify GPU
nvidia-smi

# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Start Ollama service
sudo systemctl start ollama
sudo systemctl enable ollama
# Download fine-tuned model from S3
aws s3 cp s3://25bw86-cisc886-bucket/model/25bw86-techsupport.gguf ~/25bw86-techsupport.gguf
ollama create 25bw86-techsupport -f ~/Modelfile
# Test the model
ollama run 25bw86-techsupport "My Windows computer shows a blue screen error with code 0x0000007E. What should I do?"
```

Modelfile:

Modelfile used to create custom Ollama model:

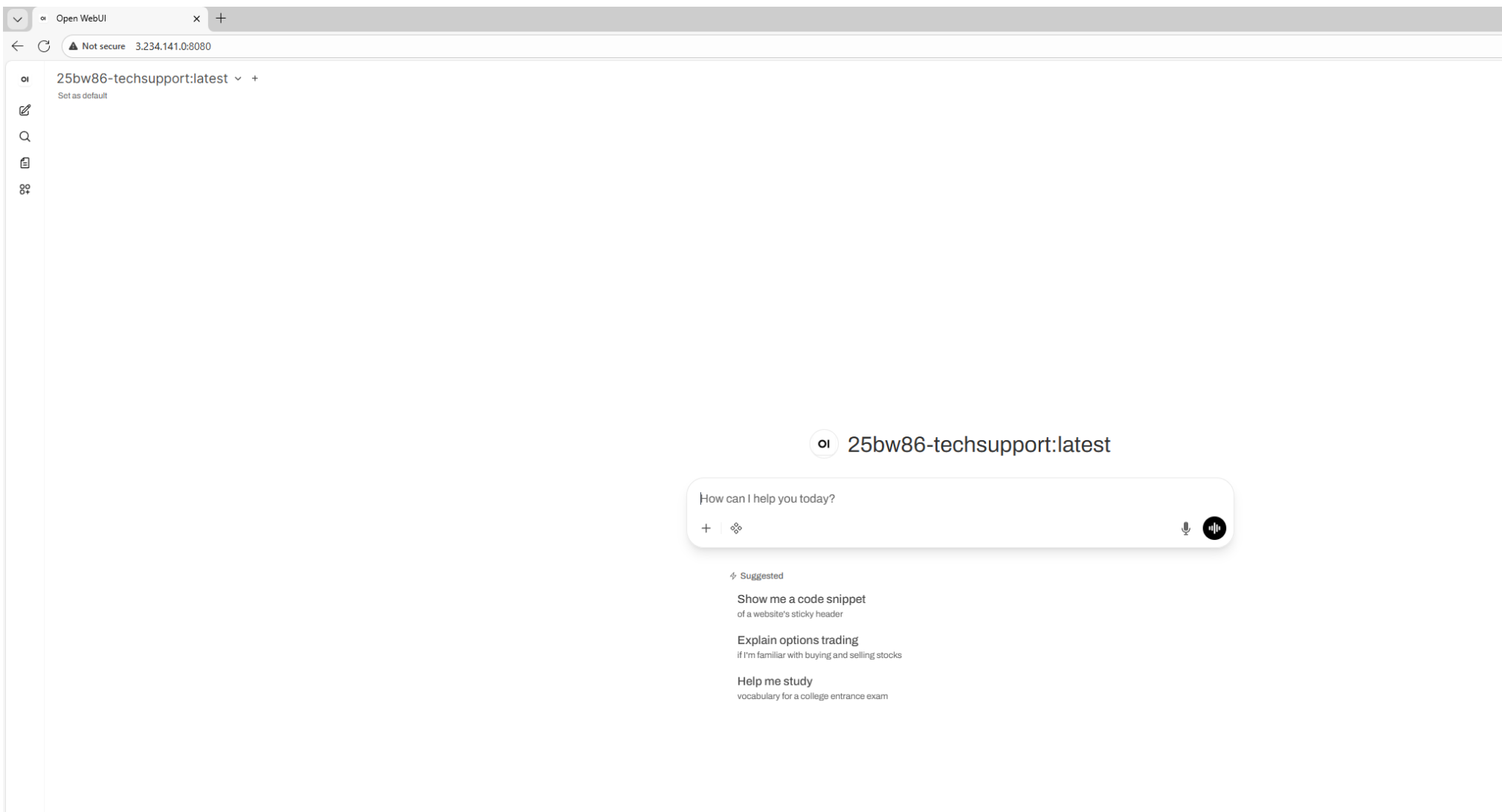
```
FROM /home/ubuntu/25bw86-techsupport.gguf

PARAMETER num_ctx 2048
PARAMETER temperature 0.7
PARAMETER repeat_penalty 1.1

SYSTEM You are a tech support assistant. Answer technical
questions helpfully and clearly.
```

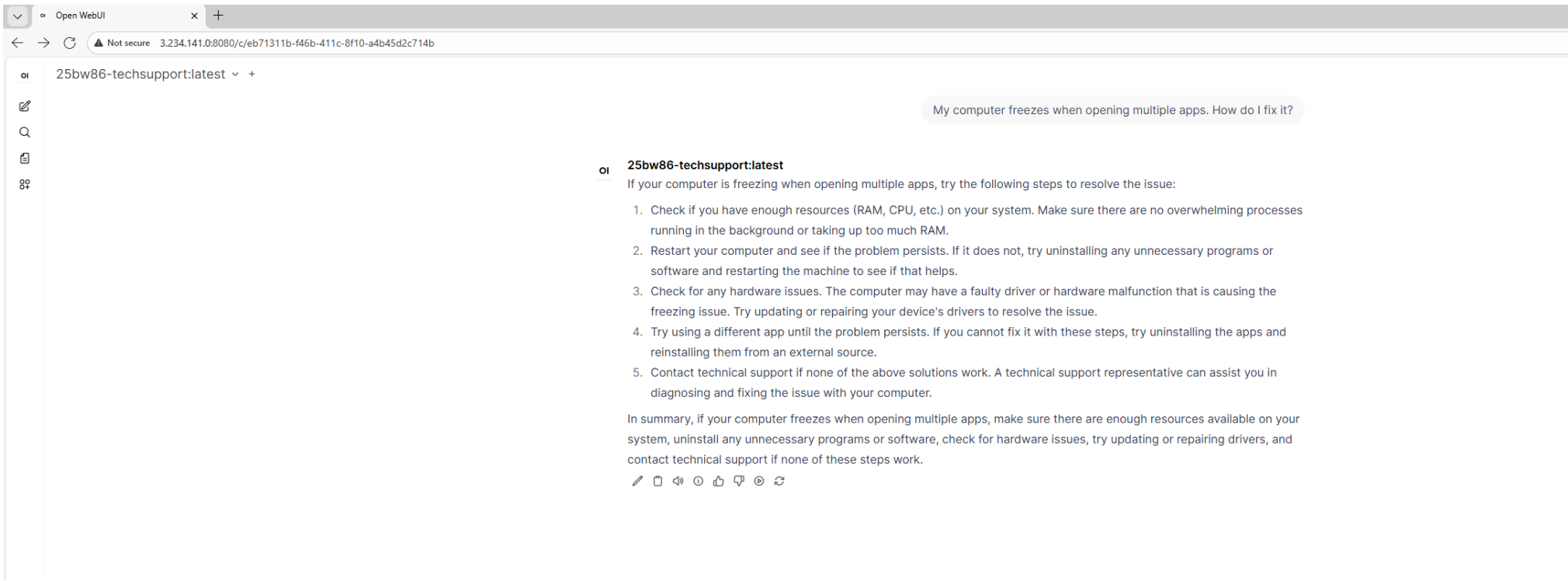
curl API Test:

```
curl http://localhost:11434/api/generate \
  --header "Content-Type: application/json" \
  --data '{
    "model": "25bw86-techsupport",
    "prompt": "My WiFi keeps disconnecting every few minutes. How do I fix it?",
    "stream": false
  }'
```

The model successfully responded with detailed WiFi troubleshooting steps, confirming the API is working correctly on the EC2 instance with GPU acceleration (Tesla T4, CUDA 12.2).

Section 7 — Web Interface





OpenWebUI Deployment:

OpenWebUI was deployed as a Docker container on the EC2 instance using the following command:

```
sudo docker run -d \
  --name 25bw86-openwebui \
  --restart always \
  --network host \
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  -p 8080:8080 \
  ghcr.io/open-webui/open-webui:main
```

The --restart always flag ensures OpenWebUI automatically restarts when the EC2 server reboots, satisfying the requirement for automatic startup. The interface is accessible at http://[EC2-PUBLIC-IP]:8080.

Note: The interface uses HTTP (not HTTPS) as SSL certificate configuration is outside the scope of this academic project. In a production deployment, HTTPS with a valid SSL certificate would be required.

Sample Conversation:

A sample conversation was conducted through the OpenWebUI interface with the fine-tuned 25bw86-techsupport model.

The user asked: "My computer freezes when opening multiple apps. How do I fix it?"

The model responded with a detailed 5-step troubleshooting guide covering resource management, restarting applications, checking for hardware issues, reinstalling apps, and contacting technical support — demonstrating the chatbot's effectiveness for tech support queries.

Cost Summary

AWS Service	Usage	Approximate Cost
EC2 g4dn.xlarge	~5 hours	~$2.67
EMR Cluster (3x m5.xlarge)	~2.5 hour	~$1.68
S3 Storage (120GB)	Full dataset + processed data	~$2.76

AWS Service	Usage	Approximate Cost
Data Transfer	Upload/Download	~$1.0
Total		~$8.84

GitHub Repository

URL: https://github.com/a7mednafea/25bw86-cisc886-project

File	Description
fine-tuning/fine_tune_notebook.ipynb	Fine-tuning notebook
fine-tuning/finetune.py	Fine-tuning Python script
spark/preprocess.py	PySpark preprocessing pipeline
spark/upload_data.py	Dataset upload script
spark/upload_full.py	Full dataset streaming upload to S3
report/figures/	Architecture diagram + EDA figures
README.md	Full replication guide + cost table
deployment/deploy.sh	EC2 deployment script